

Term Weighting

Introduction to Information Retrieval

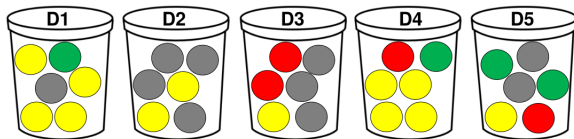
Dwaipayan Roy

Department of Computational and Data Sciences



Are all the terms equally important?

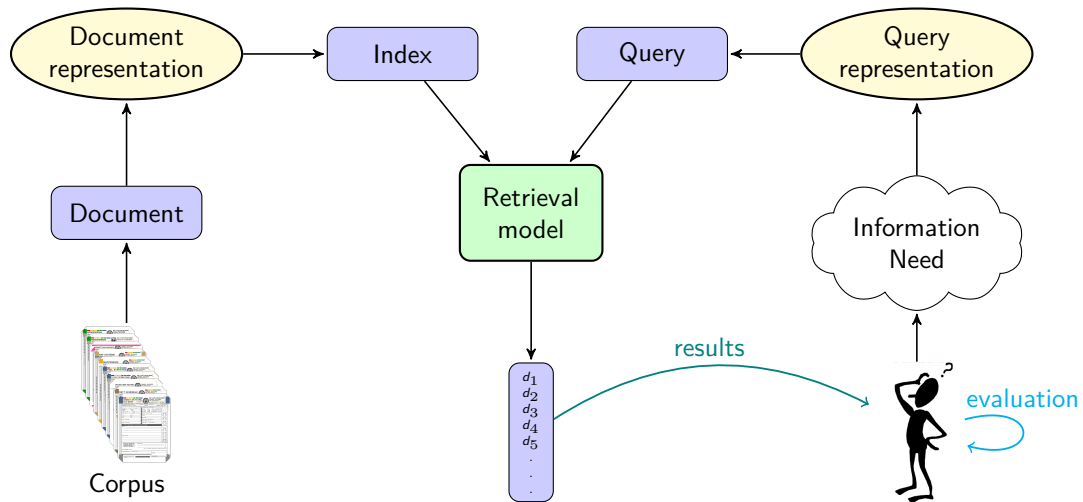
- Assume - Collection → five documents -
 - ▶ different colored ball → different terms.



- Query -   
 - ▶ Which is the most relevant document?
 - ▶ Which is the least relevant document?

We need a factor for **term importance**.

Information Retrieval: A Graphical Representation



The retrieval task

Input

- A corpus / collection C (set of documents : $d_1, d_2, \dots d_m$)
- A query $Q = \{q_1, q_2, \dots\}$

Output

- A set of documents relevant to the query Q .

The retrieval task

Input

- A corpus / collection C (set of documents : $d_1, d_2, \dots d_m$)
- A query $Q = \{q_1, q_2, \dots\}$

Output

- A set of documents relevant to the query Q .

- ✓ Boolean retrieval.
- Ranked retrieval.

Best-match retrieval models

beyond exact-match retrieval models

- Best-match models predict the degree to which a document is relevant to a query.
- $\text{relevance}(d, Q) \sim \text{similarity}(d, Q)$.

Best-match retrieval models

beyond exact-match retrieval models

- Best-match models predict the degree to which a document is relevant to a query.
- $\text{relevance}(d, Q) \sim \text{similarity}(d, Q)$.
- How to compute the similarity between d and Q ? $\rightarrow$ taken care of by retrieval models.

Best-match retrieval models

beyond exact-match retrieval models

- Best-match models predict the degree to which a document is relevant to a query.
- $\text{relevance}(d, Q) \sim \text{similarity}(d, Q)$.
- How to compute the similarity between d and Q ? $\rightarrow$ taken care of by retrieval models.
- For each d in C :
 - ▶ assign a score to document $d = \text{score}(d, Q)$

Do we really need to compute the score for all the documents?

The Naive Approach

- For each d in C :
 - ▶ assign a score to document $d = \text{score}(d, Q)$

Do we really need to compute the score for all the documents?

The Naive Approach

- For each d in C :
 - ▶ assign a score to document $d = \text{score}(d, Q)$

The Problem: Scalability

- Web-scale collections contain **billions** of documents.
- Iterating over every single one for every query is **computationally infeasible**.
- Incredibly **slow** and waste resources on documents having no chance of being relevant.

Do we really need to compute the score for all the documents?

The Naive Approach

- For each d in C :
 - ▶ assign a score to document $d = \text{score}(d, Q)$

The Problem: Scalability

- Web-scale collections contain **billions** of documents.
- Iterating over every single one for every query is **computationally infeasible**.
- Incredibly **slow** and waste resources on documents having no chance of being relevant.

The Efficient Solution: The Inverted Index to the Rescue!

We don't need to score all documents. We only need to score the **candidate set** of documents that contain *at least one query term*.

- The inverted index allows us to instantly find this candidate set.
- For a query like **quantum computing**, we only score documents from the postings lists for **quantum** and **computing**.
- This reduces the problem from $O(|V|)$ to $O(|QueryTerms|)$, where $|| \ll |V|$.

Do we really need to compute the score for all the documents?

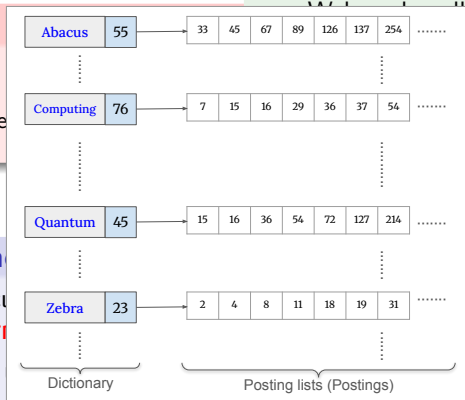
The Problem: Scalability

The Naive Approach

- For each d in C :
 - assign a score to document

The Efficient Solution: The Inverted Index

We don't need to score all documents that contain *at least one query term*



- The inverted index allows
- For a query like **quantum computing**, we only score documents from the postings lists for **quantum** and **computing**.
- This reduces the problem from $O(|V|)$ to $O(|QueryTerms|)$, where $|QueryTerms| \ll |V|$.

Sections contain **billions** of

every single one for every query
computationally infeasible.

and waste resources on
giving no chance of being relevant.

Relevant set of documents that

Scoring a document

- A document $d = \{t_1, t_2, \dots, t_M\}$

Scoring a document

- A document $d = \{t_1, t_2, \dots, t_M\}$
- Scoring a document $\rightarrow$ combined scores of the terms t_i in the document d .

Scoring a document

- A document $d = \{t_1, t_2, \dots, t_M\}$
- Scoring a document $\rightarrow$ combined scores of the terms t_i in the document d .

$$score(d, Q) = \begin{cases} \sum_{t \in d} score(t, Q) \\ \prod_{t \in d} score(t, Q) \\ \dots \end{cases}$$

Ranked retrieval

keyword matching

- Need a way of assigning a score to a query, document pair. $\text{score}(d, Q) = ?$

Ranked retrieval

keyword matching

- Need a way of assigning a score to a query, document pair. $\text{score}(d, Q) = ?$
- Consider a one-term query: $Q = \{q\}$
 - ▶ If the query term does not occur in the document ($q \notin d$): $\text{score}(d, Q) = 0$.

Ranked retrieval

keyword matching

- Need a way of assigning a score to a query, document pair. $\text{score}(d, Q) = ?$
- Consider a one-term query: $Q = \{q\}$
 - ▶ If the query term does not occur in the document ($q \notin d$): $\text{score}(d, Q) = 0$.
- The more frequent the query term in the document, the higher the score (should be).

Ranked retrieval

keyword matching

- Need a way of assigning a score to a query, document pair. $\text{score}(d, Q) = ?$
- Consider a one-term query: $Q = \{q\}$
 - ▶ If the query term does not occur in the document ($q \notin d$): $\text{score}(d, Q) = 0$.
- The more frequent the query term in the document, the higher the score (should be).
 - ▶ Does **10× more mentions** mean the document is **10× more relevant**? **No**.

Ranked retrieval

keyword matching

- Need a way of assigning a score to a query, document pair. $\text{score}(d, Q) = ?$
- Consider a one-term query: $Q = \{q\}$
 - ▶ If the query term does not occur in the document ($q \notin d$): $\text{score}(d, Q) = 0$.
- The more frequent the query term in the document, the higher the score (should be).
 - ▶ Does **10× more mentions** mean the document is **10× more relevant**? **No**.
- In Bag-of-words model:
 - ▶ Calculates the similarity based on the query - document overlap.
 - ▶ Doesn't consider the ordering of words in a document.

Ranked retrieval

keyword matching

- Need a way of assigning a score to a query, document pair. $\text{score}(d, Q) = ?$
- Consider a one-term query: $Q = \{q\}$
 - ▶ If the query term does not occur in the document ($q \notin d$): $\text{score}(d, Q) = 0$.
- The more frequent the query term in the document, the higher the score (should be).
 - ▶ Does **10× more mentions** mean the document is **10× more relevant**? **No**.
- In Bag-of-words model:
 - ▶ Calculates the similarity based on the query - document overlap.
 - ▶ Doesn't consider the ordering of words in a document.
 - ! **John is quicker than Mary** and **Mary is quicker than John** have the same representation.

Ranked retrieval

$$\text{Score}(d, Q) = \begin{cases} \sum_{t \in Q} f(d, t) \\ \prod_{t \in Q} g(d, t) \\ \dots \end{cases}$$

Query: *Vehicle Smoke Pollution*

$$\text{Score}(d, Q) = f(d, \text{Vehicle}) + f(d, \text{Smoke}) + f(d, \text{Pollution})$$

Ranked retrieval

$$\text{Score}(d,Q) = \begin{cases} \sum_{t \in Q} f(d,t) \\ \prod_{t \in Q} g(d,t) \\ \dots \end{cases}$$

Query: *Vehicle Smoke Pollution*

$$\text{Score}(d,Q) = f(d, \text{Vehicle}) + f(d, \text{Smoke}) + f(d, \text{Pollution})$$

- $f(\cdot)$, $g(\cdot)$ - term weighting functions.

Term weighting

term frequency (tf)

- $tf(t,d)$ of a term t in a document d = no. of times t occurs in document d .
- We want to use tf as a measure of importance when matching query with a document.

Term weighting

term frequency (tf)

- $tf(t, d)$ of a term t in a document d = no. of times t occurs in document d .
- We want to use tf as a measure of importance when matching query with a document.
- $tf(q, d1)=10$, $tf(q, d2)=100$
 - ▶ $relevance(d2) = 10 \times relevance(d1)$? **NO**

Term weighting

term frequency (tf)

- $tf(t, d)$ of a term t in a document d = no. of times t occurs in document d .
- We want to use tf as a measure of importance when matching query with a document.
- $tf(q, d1)=10$, $tf(q, d2)=100$
 - ▶ $relevance(d2) = 10 \times relevance(d1)$? **NO**
- Raw term frequency is not a good idea.
- Relevance does not increase proportionally with term frequency.

Term Frequency Scaling

Taming raw counts with sub-linear functions

The Problem with Raw Counts

Raw term frequency over-emphasizes long documents and has diminishing returns on relevance.

Term Frequency Scaling

Taming raw counts with sub-linear functions

The Problem with Raw Counts

Raw term frequency over-emphasizes long documents and has diminishing returns on relevance.

Log Frequency

$$w = 1 + \log_{10}(tf)$$

tf	weight
1	1.0
2	1.3
10	2.0
100	3.0

Term Frequency Scaling

Taming raw counts with sub-linear functions

The Problem with Raw Counts

Raw term frequency over-emphasizes long documents and has diminishing returns on relevance.

Log Frequency

$$w = 1 + \log_{10}(tf)$$

tf	weight
1	1.0
2	1.3
10	2.0
100	3.0

Double Log

$$w = 1 + \log(1 + \log(tf))$$

tf	weight
1	1.0
2	1.2
10	1.3
100	1.6

Term Frequency Scaling

Taming raw counts with sub-linear functions

The Problem with Raw Counts

Raw term frequency over-emphasizes long documents and has diminishing returns on relevance.

Log Frequency

$$w = 1 + \log_{10}(tf)$$

tf	weight
1	1.0
2	1.3
10	2.0
100	3.0

Double Log

$$w = 1 + \log(1 + \log(tf))$$

tf	weight
1	1.0
2	1.2
10	1.3
100	1.6

Saturation

$$w = \frac{tf}{k + tf}$$

tf	weight
1	0.5
2	0.7
10	0.9
100	1.0

(k=1)

Term frequency

scaling term frequency

- The log frequency weight of term t in d is defined as follows:



$$w_{t,d} = \begin{cases} 1 + \log_{10} \text{tf}(t,d) & \text{if } \text{tf}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

- $\text{tf}(t, d) \Rightarrow w(t, d) : 0 \Rightarrow 0, 1 \Rightarrow 1, 2 \Rightarrow 1.3, 10 \Rightarrow 2, 1000 \Rightarrow 4$ etc.
- Score for a Query-Document pair: sum over terms q in both Q and d :

$$\text{tf-matching-score}(Q, d) = \sum_{q \in Q} (1 + \log_{10} \text{tf}(q, d))$$

- The score is 0 if none of the query terms is present in the document.

Extra importance to rare terms

- Rare, uncommon terms are more informative than frequent terms.
- Consider the query:

diseases caused by smoking

- ▶ diseases, smoking - rare in the collection as compared to cause .
- A document containing this term is very likely to be relevant.

Extra importance to rare terms

- Rare, uncommon terms are more informative than frequent terms.
- Consider the query:

diseases caused by smoking

- ▶ diseases, smoking - rare in the collection as compared to cause .
- A document containing this term is very likely to be relevant.
 - ▶ diseases smoking vs. smoking cause or cause diseases

Extra importance to rare terms

- Rare, uncommon terms are more informative than frequent terms.
- Consider the query:

diseases caused by smoking

- ▶ diseases, smoking - rare in the collection as compared to cause .
- A document containing this term is very likely to be relevant.
 - ▶ diseases smoking vs. smoking cause or cause diseases
- High weights for rare terms.

Need a factor to quantify the rareness of query terms.

What about common terms?

term weights

- $tf(t,d) \rightarrow \text{high}$, when $t \rightarrow \text{common term}$.

What about common terms?

term weights

- $tf(t,d)$ $\rightarrow$ high, when t $\rightarrow$ common term.
- Common terms are less informative than rare terms.
- Consider a term in the query that is frequent in the collection (e.g., **cause**).

What about common terms?

term weights

- $tf(t,d)$ $\rightarrow$ high, when t $\rightarrow$ common term.
- Common terms are less informative than rare terms.
- Consider a term in the query that is frequent in the collection (e.g., **cause**).
- A document containing this term is more likely to be relevant than a document that doesn't ...
- ... but these words are not sure indicators of relevance.

What about common terms?

term weights

- $tf(t,d)$ $\rightarrow$ high, when t $\rightarrow$ common term.
- Common terms are less informative than rare terms.
- Consider a term in the query that is frequent in the collection (e.g., **cause**).
- A document containing this term is more likely to be relevant than a document that doesn't ...
- ... but these words are not sure indicators of relevance.
- For frequent terms, we want positive weights ...
- ... but lower weights than for rare terms.

Document frequency

indicator of importance

- We want **high** weights for rare terms.
- We want **low** (but positive) weights for frequent words.

Document frequency

indicator of importance

- We want **high** weights for rare terms.
- We want **low** (but positive) weights for frequent words.
- We will use *document frequency* to factor this into computing the matching score.

Document frequency

indicator of importance

- We want **high** weights for rare terms.
- We want **low** (but positive) weights for frequent words.
- We will use *document frequency* to factor this into computing the matching score.

Document Frequency - $df(t)$

- Number of documents in the collection that contains the term t .

Inverse document frequency

indicator of importance

- $df(t)$ is the document frequency, the number of documents that t occurs in.
- $df(t)$ is an inverse measure of the informativeness of term t .
- We define the idf weight of term t as follows:

$$idf(t) = \log_{10} \frac{N}{df_t}$$

- ▶ $N \Rightarrow$ number of documents in the collection
- $idf(t)$: measure of the informativeness of t

IDF

examples

$$idf(t) = \log_{10} \frac{N}{df_t} (N = 1,000,000)$$

term	$df(t)$	$idf(t)$
quantum	1	6
animal	100	4
sunday	1000	3
fly	10000	2
cause	100000	1
the	1000000	0

TF-IDF term weighting

- Increases with the number of occurrences within a document
- Increases with the rarity of the term in the collection
- Combines TF and IDF to find the weight of terms:

$$w(t, d) = (1 + \log_{10} tf(t, d)) \cdot idf(t)$$

TF-IDF term weighting

- Increases with the number of occurrences within a document
- Increases with the rarity of the term in the collection
- Combines TF and IDF to find the weight of terms:

$$w(t, d) = (1 + \log_{10} tf(t, d)) \cdot idf(t)$$

- For a query Q and document d , retrieval `score(d, Q)` :

$$score(d, Q) = \sum_{t \in Q} w(t, d)$$

TF-IDF term weighting

- Increases with the number of occurrences within a document
- Increases with the rarity of the term in the collection
- Combines TF and IDF to find the weight of terms:

$$w(t, d) = (1 + \log_{10} tf(t, d)) \cdot idf(t)$$

- For a query Q and document d , retrieval `score(d, Q)` :

$$score(d, Q) = \sum_{t \in Q} w(t, d)$$

Can we reduce the summation over a small number of terms?

TF-IDF term weighting

advanced incidence matrix

	Inception	The Dark Knight	Interstellar	Dunkirk	The Prestige	Memento
duality	0	1	0	0	1	0
identity	0	1	0	0	1	1
memory	1	0	0	0	0	1
rivalry	0	1	0	0	1	0
sacrifice	0	0	1	1	1	0
survival	0	0	1	1	0	0
time	1	0	1	1	0	1

TF-IDF term weighting

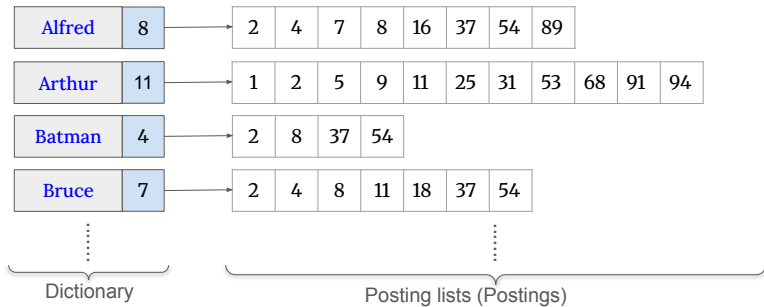
advanced incidence matrix

	Inception	The Dark Knight	Interstellar	Dunkirk	The Prestige	Memento
duality	0	1	0	0	1	0
identity	0	1	0	0	1	1
memory	1	0	0	0	0	1
rivalry	0	1	0	0	1	0
sacrifice	0	0	1	1	1	0
survival	0	0	1	1	0	0
time	1	0	1	1	0	1

- Store the TF-IDF weights instead of the binary indicator.

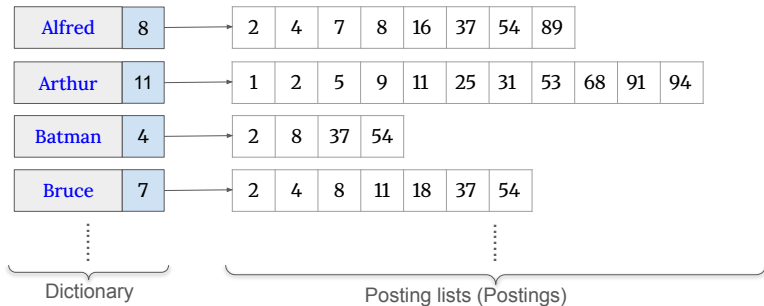
TF-IDF term weighting

inverted index



TF-IDF term weighting

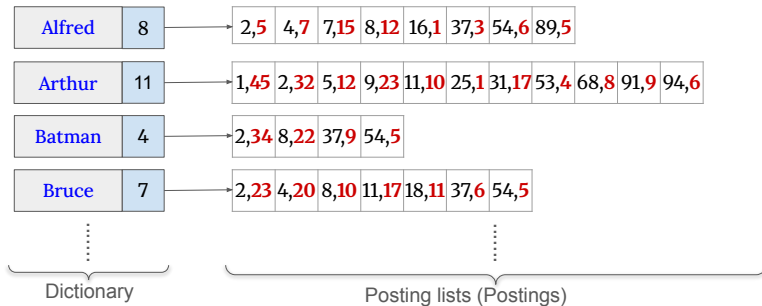
inverted index



- Store the TF instead of the binary indicator.

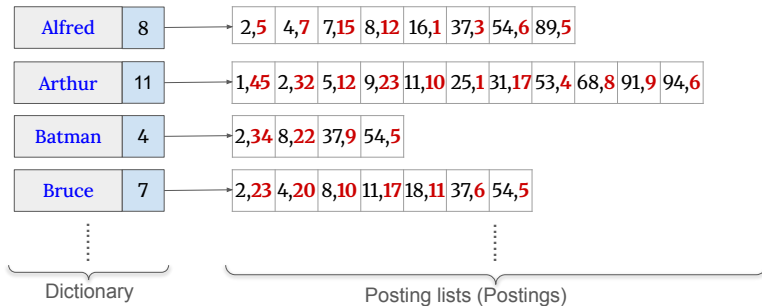
TF-IDF term weighting

inverted index



TF-IDF term weighting

inverted index



- Store the TF-IDF weights instead of the TF.